

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..40

LOGGING

 Copy as Markdown

Logging

Logging is how you find out what your server did after it did it: which calls ran, which queries were slow, and what failed. Serverpod records this for you, and you add your own messages on top.

There are three kinds of record, and the difference matters for everything below:

- **Session records** describe one unit of work, such as an endpoint call: how long it took, whether it failed, and which endpoint it hit.
- **Query records** describe individual database queries run during that session.
- **Log messages** are the entries you write yourself with `session.log`.

Write your own messages

Call `log` on the `session` you were given:

```
session.log('This is working well');
```

Pass a level, an exception, and a stack trace when something goes wrong:

```
session.log(  
  'Oops, something went wrong',  
  level: LogLevel.warning,  
  exception: e,  
  stackTrace: stackTrace,  
);
```

Messages are collected while the session runs and written when it closes, whether it finished normally or threw.



Ask AI

Where logs go

Records are written to the database, to the console, to both, or to neither.

In the database they land in three tables:

Table	Holds
<code>serverpod_session_log</code>	One row per completed session.
<code>serverpod_log</code>	Your <code>session.log</code> messages.
<code>serverpod_query_log</code>	Database queries.

The last two reference the session row, so deleting a session row removes its queries with it.

INFO

The companion app [Serverpod Insights](#) reads and searches these tables, and can change the runtime settings described below.

Which sessions get recorded

Not every session produces a row, and the default depends on the run mode.

In `development`, every completed session is recorded. In `staging`, `production`, and `test`, a session is recorded only when it ran longer than one second, it failed, or it produced a log or query entry. Ordinary fast calls leave no row, which keeps the table to the sessions worth looking at.

These thresholds are runtime settings stored in the `serverpod_runtime_settings` table, so you can change them on a running server through Insights without redeploying. They control whether all sessions are logged, whether all queries are logged, what counts as slow, and the minimum level a message must have to be kept. You can also override them per endpoint and per method.

Configure logging

Session logging is configured under `sessionLogs:` in your config file for the run mode, or through environment variables. Environment variables win over the config file, key by key.

Setting	Environment variable	Default
<code>persistentEnabled</code>	<code>SERVERPOD_SESSION_PERSISTENT_LOG_ENABLED</code>	<code>true</code> when a database is configured
<code>consoleEnabled</code>	<code>SERVERPOD_SESSION_CONSOLE_LOG_ENABLED</code>	<code>true</code> in <code>development</code> or when there is no database, otherwise <code>false</code>
<code>consoleLogFormat</code>	<code>SERVERPOD_SESSION_CONSOLE_LOG_FORMAT</code>	<code>text</code> in <code>development</code> , otherwise <code>json</code>

Three more settings control [purging](#), which behaves differently enough to be worth reading before you rely on it.

```
sessionLogs:
  persistentEnabled: true    # Store records in the database
  consoleEnabled: true      # Also print them
  cleanupInterval: 6h       # Purge every 6 hours
  retentionPeriod: 30d      # Keep 30 days
  retentionCount: 5000      # Keep at most 5,000 sessions
```

Durations use the same format as [model default values](#), such as `30d`, `6h`, or `1d 2h 30min`.

⚠ WARNING

Setting `persistentEnabled` to `true` without a configured database throws a `StateError` on startup. Persistent logging needs somewhere to persist to.

⚠ WARNING

Persistent logging is unavailable on SQLite, which cannot handle the concurrent writes it needs. The server warns and skips it, and nothing takes its place, so enable `consoleEnabled` if you want records in the run modes where it defaults to off.

ℹ INFO

Every environment variable in the table takes a real value. Setting one to an empty string is not a way to unset it: the server fails to start. To turn a policy off, set the key to `null` in the config file instead.

Purge old records

Log tables grow with every call your server handles, so Serverpod can delete old records for you. Cleanup runs on the `cleanupInterval`, and removes session rows that are either older than `retentionPeriod` or beyond the newest `retentionCount`, whichever applies first. Deleting a session row takes its query and log rows with it.

Setting	Environment variable	Default
<code>cleanupInterval</code>	<code>SERVERPOD_SESSION_LOG_CLEANUP_INTERVAL</code>	Unset, so no purging
<code>retentionPeriod</code>	<code>SERVERPOD_SESSION_LOG_RETENTION_PERIOD</code>	Unset, so no age limit
<code>retentionCount</code>	<code>SERVERPOD_SESSION_LOG_RETENTION_COUNT</code>	Unset, so no count limit

⚠ WARNING

These three fall back to `24h`, `90d`, and `100000` only when `sessionLogs` is absent from your config entirely. Set any one session-log key, in the file or through an environment variable, and the ones you did not set resolve to unset rather than to those values, switching that policy off with no warning.

Generated projects ship a `sessionLogs` block in the `development`, `test`, and `production` configs, so purging is off in those run modes until you set all three explicitly. Set `cleanupInterval`, `retentionPeriod`, and `retentionCount` together whenever you configure any part of `sessionLogs`.

The `cleanupInterval` setting is the switch for the whole job. With no interval set, nothing is purged whatever the two retention values say. Purging also requires `persistentEnabled`, since there is nothing to delete otherwise.

Cleanup is triggered by log writes rather than by a timer, so a server that is not logging anything does not purge. A single pass gives up after an hour, and the next interval starts a fresh one.

Related

- [Configuration](#): the config files these settings live in.
- [Configuration reference](#): every session-log key with its environment variable.
- [Sessions](#): what a session is, and the `log` method.
- [Insights](#): reading logs and editing runtime settings.
- [Load testing](#): why persisting every session during a run becomes the bottleneck.